```
  @PostMapping
  public Mono<Order> submitOrder(
    @RequestBody @Valid OrderRequest orderRequest     ❹
  ) {
    return orderService.submitOrder(
     orderRequest.isbn(), orderRequest.quantity()
    );
  }
}
```

❶ Stereotype annotation marking a class as a Spring component and a source of handlers for REST endpoints

❷ Identifies the root path mapping URI for which the class provides handlers (/orders)

❸ A Flux is used to publish multiple orders (0..N).

❹ Accepts an OrderRequest object, validated and used to create an order. The created order is returned as a Mono.

This REST controller completes our basic implementation of the Order Service application. Let's see it in action. First, make sure the PostgreSQL container you created earlier is still running. Then open a Terminal window, navigate to the Order Service project's root folder, and run the application:

```
$ ./gradlew bootRun
```

You can try out the API by submitting an order. The application will save the order as rejected and return a `200` response to the client:

```
$ http POST :9002/orders isbn=1234567890 quantity=3

HTTP/1.1 200 OK
{
  "bookIsbn": "1234567890",
  "bookName": null,
  "bookPrice": null,
  "createdDate": "2022-06-06T09:40:58.374348Z",
  "id": 1,
  "lastModifiedDate": "2022-06-06T09:40:58.374348Z",
  "quantity": 3,
  "status": "REJECTED",
  "version": 1
}
```